

Smart DMS v2.0 — Document Management Portal

****Company:**** ZF Rane Automotive India Private Limited

****Stack:**** Python · Flask · SQLite · Jinja2 · Vanilla JS

Table of Contents

1. [Quick Start](#quick-start)
2. [Demo Credentials](#demo-credentials)
3. [Plants & Locations](#plants--locations)
4. [Project Structure](#project-structure)
5. [Pages & How They Work](#pages--how-they-work)
 - [Login](#1-login---)
 - [Register](#2-register---)
 - [Forgot / Reset Password](#3-forgot--reset-password)
 - [Dashboard](#4-dashboard---)
 - [Upload Documents](#5-upload-documents---)
 - [Pending Items (Approvals)](#6-pending-items-approvals---)
 - [Approval Review](#7-approval-review)
 - [Document Library](#8-document-library---)
 - [Master Records (Plant Assets)](#9-master-records-plant-assets---)
 - [Customer Records](#10-customer-records---)
 - [Graphics Report](#11-graphics-report---)
 - [Revision History](#12-revision-history---)
 - [Archive](#13-archive---)
 - [System Log](#14-system-log---)
 - [People](#15-people---)
 - [Profile](#16-profile---)
 - [About DMS](#17-about-dms)
 - [About Company](#18-about-company)
6. [Role & Access Control](#role--access-control)
7. [Database Tables](#database-tables)
8. [Services Reference](#services-reference)
9. [Environment Variables](#environment-variables)
10. [Standard Operating Procedure](#standard-operating-procedure)
11. [Dependencies](#dependencies)

Quick Start

1. Create and activate virtual environment

```
python -m venv venv
```

```
venv\Scripts\activate # Windows
```

```
source venv/bin/activate # macOS / Linux
```

2. Install dependencies

```
pip install -r requirements.txt
```

3. Run

```
python app.py
```

Open **`http://localhost:5001`**

Demo Credentials

You can also log in using the **Employee ID (GENID)** instead of email — e.g. ``EMP001` / `Pass@12345``.

Plants & Locations

Project Structure

dms_portal/

■■■■ app.py # Flask app factory, blueprint registration

■■■■ config.py # All config via environment variables

■■■■ database.py # SQLite schema creation and migrations

■■■■ extensions.py # Flask-Mail instance

■■■■ requirements.txt

■■■■ smart_dms.db # Main SQLite database (auto-created)

■

■■■■ data/

■ ■■■■ mock_data.py # Seed data: users, plants, customers, records

■ ■■■■ departments.py # Official department list + normalization

■ ■■■■ customers.py # Official customer list + normalization

■ ■■■■ document_library_data.py # Static library category tree

■ ■■■ smart_dms_users.sqlite3 # User credentials database

■

■■■ routes/ # One Blueprint per page

■ ■■■ auth_routes.py

■ ■■■ dashboard_routes.py

■ ■■■ upload_routes.py

■ ■■■ approval_routes.py

■ ■■■ archive_routes.py

■ ■■■ document_library_routes.py

■ ■■■ plant_assets_routes.py

■ ■■■ customer_records_routes.py

■ ■■■ graphics_report_routes.py

■ ■■■ revision_history_routes.py

■ ■■■ system_log_routes.py

■ ■■■ people_routes.py

■ ■■■ profile_routes.py

■ ■■■ notification_routes.py

■ ■■■ about_routes.py

■ ■■■ category_routes.py

■

■■■ services/ # Business logic, one class per domain

■ ■■■ auth_service.py

■ ■■■ document_service.py

■ ■■■ document_library_service.py

■ ■■■ category_document_service.py

■ ■■■ plant_asset_service.py

■ ■■■ customer_record_service.py

■ ■■■ revision_history_service.py

■ ■■■ document_tracking_service.py

■ ■■■ document_preview_service.py

■ ■■■ pdf_conversion_service.py

■ ■■■ system_log_service.py

■ ■■■ notification_service.py

■ ■■■ mail_service.py

■ ■■■ password_reset_service.py

■ ■■■ user_store_service.py

■

- templates/
- ■■■ layout.html # Shared shell: sidebar + topbar + alerts
- ■■■ components/
- ■ ■■■ sidebar.html # Navigation sidebar
- ■■■ auth/
- ■ ■■■ login.html
- ■ ■■■ register.html
- ■ ■■■ reset_password.html
- ■■■ dashboard.html
- ■■■ upload.html
- ■■■ approvals.html
- ■■■ approval_review.html
- ■■■ document_library.html
- ■■■ document_view.html
- ■■■ plant_assets.html
- ■■■ customer_records.html
- ■■■ graphics_report.html
- ■■■ revision_history.html
- ■■■ archive.html
- ■■■ system_log.html
- ■■■ people.html
- ■■■ profile.html
- ■■■ about.html
- ■■■ about_company.html
-
- static/
- ■■■ css/app.css # Dark industrial theme (IBM Plex fonts)
- ■■■ js/
- ■■■ app.js # Alert auto-dismiss, sidebar toggle, theme
- ■■■ upload.js # Drag-and-drop, target switching, validation
- ■■■ plant_assets.js # Plant → dept → files AJAX
- ■■■ customer_records.js # Customer → files AJAX
- ■■■ document_library.js # Category browser AJAX
-
- uploads/ # All uploaded files stored here

Pages & How They Work

1. Login — `/` or `/login`

****Route:**** `auth\_routes.py` → `login()`

The entry point of the portal. Accepts either an ****email address**** or an ****Employee ID (GENID)**** as the username — the system auto-detects which one was entered based on whether it contains `@`.

On successful login, the user's name, email, ID, plant, department, and role are stored in the Flask session.

A login event is written to the System Log with the user's IP address.

If already logged in, the user is redirected directly to the Dashboard.

A "Forgot Password" link on this page triggers a token-based reset email.

2. Register — `/register`

****Route:**** `auth\_routes.py` → `register()`

Self-service account creation. New users fill in:

All new accounts are assigned the ****User**** role automatically. An admin must manually promote a user to Manager, Approver, or Admin via the database. A register event is written to the System Log.

3. Forgot / Reset Password

****Routes:**** `auth\_routes.py` → `forgot\_password()` / `reset\_password()`

User submits their email on the login page.

If the email exists, a signed time-limited token is generated and a reset link is emailed via Flask-Mail.

If SMTP is not configured, the reset link is printed to the terminal (development mode).

The reset link opens a form where the user sets a new password (minimum 8 characters).

Tokens expire after 1 hour.

4. Dashboard — `/dashboard`

****Route:**** `dashboard\_routes.py` → `index()`

The main control centre. Shows all uploaded documents in a searchable, filterable table.

****Features:****

All data is loaded from the `documents` table in `smart\_dms.db`. If the database is empty, mock seed data is shown as a placeholder.

5. Upload Documents — `/upload`

****Route:**** `upload\_routes.py` → `index()`

A single upload workflow for submitting documents to the Document Library and moving them into approval.

****Key behavior:****

The page now supports one upload path only: all documents are stored in the Document Library.

Users must pick the correct category and folder path before submitting.

Every upload is automatically set to `Pending` and sent for approval.

****File Selection****

Drag-and-drop zone or click-to-browse. Accepts `.pdf`, `.docx`, `.doc`, `.xlsx`, `.xls`, `.pptx`, `.ppt`. Maximum 100 MB per file. Multiple files can be selected at once.

****Document Details****

****Revision Summary (optional)****

A toggle reveals a textarea to describe revision changes. When enabled, a revision history entry is added automatically.

****After submission:****

1. File is saved to `uploads/` with a unique timestamped filename.
2. A document record is inserted into the database with status `Pending`.
3. The upload is linked to the chosen library category and folder.
4. A confirmation email is sent to the uploader.
5. An approval request email with a secure review link is sent to the configured approver.
6. In-app notifications are created for the uploader and admin/approvers.
7. The upload is logged in the System Log.

6. Pending Items (Approvals) — `/approvals`

****Route:**** `approval\_routes.py` → `index()`

Lists all documents in the system with their approval status. Accessible to all logged-in users.

****Features:****

7. Approval Review

****Route:**** `approval\_routes.py` → `review\_document(token)`

Opened via a secure token link (from email or the Pending Items page). Shows the full document metadata and an inline PDF preview.

****Admins**** see Approve and Reject buttons. Rejection requires a comment.

****Non-admins**** can view the document and its metadata but cannot take action.

After a decision, the uploader receives an email notification and an in-app notification.

The decision is logged in the System Log.

The review token is signed with ``itsdangerous`` — it cannot be forged or tampered with.

8. Document Library — ``/document-library``

****Route:**** ``document_library_routes.py` → `index()``

A structured, category-based file browser for reference documents. Unlike the Dashboard (which shows uploaded submissions), the Document Library organises files into a predefined category tree defined in ``data/document_library_data.py``.

****Categories include:**** Procedures, CQ Manuals, Plant Procedures, Awards & Certifications, and more.

Navigation works by selecting a category from the left panel, then drilling into primary and secondary subcategories. Files can be viewed inline. All views are logged in the System Log.

9. Master Records (Plant Assets) — ``/plant-assets``

****Route:**** ``plant_assets_routes.py` → `index()``

Browse documents organised by ****Plant → Department****. The page loads plant cards; clicking a plant card fetches its departments via AJAX; clicking a department tab fetches the file list for that plant/department combination.

All data is loaded dynamically via AJAX (``/plant-assets/departments`` and ``/plant-assets/files``).

File views are logged in the System Log.

The file list is read-only — no upload or delete from this page.

10. Customer Records — ``/customer-records``

****Route:**** ``customer_records_routes.py` → `index()``

Browse documents organised by ****Customer****. Works the same way as Master Records but grouped by customer (Hyundai, Tata, Ashok Leyland, TVS Motors, etc.) instead of plant.

Customer list is loaded on page render.

Clicking a customer card fetches its files via AJAX (``/customer-records/files``).

File views are logged in the System Log.

11. Graphics Report — `/graphics-report``

****Route:**** ``graphics_report_routes.py` → `index()`

Visual analytics dashboard with Chart.js bar and doughnut charts.

****Charts shown:****

Summary stat cards below the charts show total documents, this month's uploads, and this week's uploads. All data is pulled live from the ``documents`` table.

12. Revision History — `/revision-history``

****Route:**** ``revision_history_routes.py` → `index()`

A chronological log of all document revisions. Each entry records:

Filterable by Plant and Department. Revision entries are created automatically when a document is uploaded with the "Add revision summary" toggle enabled, or when an admin uploads a new version of an existing document.

13. Archive — `/archive``

****Route:**** ``archive_routes.py` → `index()`

****Access:**** Admin, Manager, Supervisor, Approver only.

When a document is deleted from the Dashboard, it is not permanently removed — it is moved to the Archive table with its metadata preserved. The Archive page lists all soft-deleted documents.

****Permanent Delete**** button removes the record from the archive entirely and logs the action.

Regular users cannot access this page — they are redirected to the Dashboard.

14. System Log — `/system-log``

****Route:**** ``system_log_routes.py` → `index()`

****Access:**** Admin, Manager, Supervisor, Approver only.

A full audit trail of every significant action in the system.

****Logged actions:****

Each log entry records: timestamp, user name, user ID, action type, and a details string. The log is filterable by action type using tab buttons at the top of the page.

15. People — `/people`

****Route:**** `people\_routes.py` → `index()`

****Access:**** Admin, Manager, Supervisor, Approver only.

A directory of all registered users in the system. Displays each user's:

Profile photo (or initial avatar if no photo uploaded)

Full name

Employee ID

Email address

Plant

Department

Role (with colour-coded badge)

Account creation date

A live search box filters the table client-side by any field (name, email, role, department, etc.).

16. Profile — `/profile`

****Route:**** `profile\_routes.py` → `index()`

Each logged-in user's personal page. Shows their account details (read-only) and two action panels:

****Profile Photo****

Upload a new photo (JPG, PNG, GIF, WebP).

Photo is saved to `static/avatars/` named by user ID.

Appears in the topbar and on the People page.

****Change Password****

Hidden by default, revealed by a toggle button.

Requires current password verification before accepting a new one.

New password must be at least 8 characters.

Password change is logged in the System Log.

****Activity Log****

Right panel shows all System Log entries for the current user — their personal audit trail.

17. About DMS — `/about`

****Route:**** `about\_routes.py` → `index()`

An informational page explaining what Smart DMS is, how to use it, and a guided tour of all features. Includes a step-by-step walkthrough of the upload → approval → archive workflow.

18. About Company — `/about/company`

****Route:**** `about_routes.py` → `company()`

A detailed company profile page for **ZF Rane Automotive India Private Limited**, including:

Company overview and CIN

Division details (SGD — Steering Gear Division, OSD — Occupant Safety Division)

Plant locations and products

Customer list

Certifications (IATF 16949, ISO 14001, OHSAS 18001, FORD Q1)

Awards (Deming Prize, Japan Quality Medal, CII Kaizen Gold)

Engineering tools (CATIA, SolidWorks, COSMOS, AMESim)

Role & Access Control

All users can view documents from all plants and departments. Department-based filtering is available as a UI convenience but does not restrict access.

Database Tables

All tables live in `smart_dms.db`. User credentials are stored separately in `data/smart_dms_users.sqlite3`.

Services Reference

Environment Variables

All variables are optional — defaults work for local development.

If mail is not configured, password reset links are printed to the terminal.

MySQL setup and migration

Copy `.env.example` to `.env`, enter the MySQL username and password, and keep `DATABASE_ENGINE=mysql`. The application creates the configured schema and all

required tables when the MySQL account has database-creation permission.

To copy all existing SQLite records—including users, QMS levels, documents, approval history, notifications, revisions, bookmarks, and system logs—run:

```
python migrate_to_mysql.py
```

Use ``python migrate_to_mysql.py --replace`` only when the destination tables should be cleared before importing the SQLite records.

Standard Operating Procedure

A companion operational guide is available in
STANDARD_OPERATING_PROCEDURE.md.

Dependencies

Install all with:

```
pip install -r requirements.txt
```